

# UserSetupApplication

## RBAC Developer Manual

Spring Boot 4.0.1

Spring Security 7.0.2

PostgreSQL 13

JWT · jjwt 0.11.5

Version 1.0 · March 2026

Base URL: <http://localhost:8082>

# Table of Contents

---

1. Overview of RBAC
2. Architecture
3. Authentication
4. Managing Users
5. Managing Roles
6. Managing Permissions
7. API Endpoint Reference
8. Error Responses

# 1. Overview of RBAC

Role-Based Access Control (RBAC) is a security model that restricts system access based on the roles assigned to users. In UserSetupApplication, every action is guarded by a named permission. Permissions are grouped into roles, and roles are assigned to users. Users can also hold direct permissions outside of any role.

## Key Concepts

**Permission** — A named action a user can perform. Examples: `create_user`, `view_invoice`, `delete_role`.

Permissions can be individually enabled or disabled. Disabling a permission blocks all users from that action immediately — whether it's a direct or role-based permission.

**Role** — A named collection of permissions. Examples: Accountant, Super Admin, Manager. Roles can be enabled or disabled. Disabling a role removes all its permissions from assigned users at runtime without deleting any data.

**User** — A system user assigned one or more roles. Users can also have direct permissions assigned outside of roles. A disabled user cannot log in.

## How Permission Checks Work

Every protected endpoint is annotated with `@RequirePermission`. Before the request reaches the controller, the `PermissionInterceptor` checks if the current user has the required permission through two checks:

1. The user's **direct permissions** (ACTIVE only)
2. Permissions from the user's **ACTIVE roles** (ACTIVE permissions only)

If either check passes, access is granted. Otherwise a 403 Forbidden is returned.

## Request Flow

```
Request → JwtAuthenticationFilter → PermissionInterceptor → Controller
(validates token) (checks permission)
```

## Data Model

```
users
  ■■■ user_roles (join) ■■■■■→ roles
  ■■■ role_permissions (join) ■■→ permissions
  ■■■ user_permissions (join) ■■→ permissions
```

## 2. Architecture

### Security Filter Chain

| Component                | Responsibility                                                                              |
|--------------------------|---------------------------------------------------------------------------------------------|
| JwtAuthenticationFilter  | Extracts Bearer token, validates it, loads UserPrincipal into SecurityContext               |
| CustomUserDetailsService | Loads user by ID from DB, builds authorities (roles + permissions)                          |
| UserPrincipal            | Wraps User entity, provides pre-loaded authorities to avoid lazy loading                    |
| PermissionInterceptor    | AOP aspect — intercepts @RequirePermission, delegates to PermissionService                  |
| PermissionService        | Checks current user's permissions, respects ACTIVE/DISABLED status on roles and permissions |
| SecurityUtils            | Extracts UserPrincipal from SecurityContextHolder for use in services                       |

### Token Strategy

| Token         | Lifetime   | Storage              | Purpose                                                 |
|---------------|------------|----------------------|---------------------------------------------------------|
| Access Token  | 15 minutes | Client memory        | Sent as Authorization: Bearer                           |
| Refresh Token | 7 days     | DB + httpOnly cookie | Used to obtain a new access token via /api/auth/refresh |

## 3. Authentication

---

### Login

#### POST /api/auth/login

Request body:

```
{
  "email": "admin@usersetupapplication.com",
  "password": "Admin@1234"
}
```

Response:

```
{
  "accessToken": "eyJhbGciOiJI...",
  "userId": "1",
  "email": "admin@usersetupapplication.com",
  "firstName": "Admin",
  "lastName": "User",
  "roles": [
    {
      "name": "Super Admin",
      "permissions": ["create_user", "delete_user", "view_invoice"]
    }
  ],
  "directPermissions": ["export_report"]
}
```

- The refreshToken is set automatically as an httpOnly cookie.
- The accessToken must be sent as a Bearer token on all protected requests.

### Logout

#### POST /api/auth/logout

No body required. Revokes the refresh token in the database and clears the httpOnly cookie.

### Refresh Access Token

#### POST /api/auth/refresh

No body required. Reads the refreshToken from the httpOnly cookie and returns a new accessToken.

```
{
  "accessToken": "eyJhbGciOiJI..."
}
```

## 4. Managing Users

---

### Create User

**POST /users** · Permission: create\_user

```
{
  "email": "john@example.com",
  "password": "Pass@1234",
  "firstName": "John",
  "lastName": "Doe",
  "roleIds": [1],
  "permissionIds": [3, 5]
}
```

- roleIds — IDs from the roles table. At least one recommended.
- permissionIds — Direct permissions outside of roles. Can be empty [].

### Get All Users

**GET /users?page=0&size=20&sort=firstName,asc** · Permission: view\_users

Returns a paginated list of users with their roles and permissions.

### Get User by ID

**GET /users/{id}** · Permission: view\_user

Returns the full user profile including roles (with permissions) and direct permissions.

### Update User

**PUT /users/{id}** · Permission: update\_user

```
{
  "firstName": "Jane",
  "lastName": "Doe",
  "email": "jane@example.com",
  "status": "ACTIVE",
  "roleIds": [1, 2],
  "permissionIds": []
}
```

All fields are optional — only include what you want to change.

### Toggle User Status

**DELETE /users/{id}** · Permission: delete\_user

- ACTIVE → DISABLED: User cannot log in.
- DISABLED → ACTIVE: User can log in again.
- Roles and permissions are preserved — not removed on disable.

## 5. Managing Roles

*Hard deletion of roles is intentionally not supported. To prevent fraud and maintain audit integrity in the accounting system, roles can only be enabled or disabled. All role assignments and permission mappings are preserved in the database at all times.*

### Create Role

**POST /roles** · Permission: **create\_role**

```
{
  "name": "Accountant",
  "guardName": "web",
  "permissionIds": [1, 2, 3]
}
```

- guardName is optional — defaults to "web".
- permissionIds can be empty [].

### Update Role

**PUT /roles/{id}** · Permission: **edit\_role**

```
{
  "name": "Senior Accountant",
  "permissionIds": [1, 2, 3, 4]
}
```

Both fields are optional. Sending permissionIds replaces the existing permissions entirely.

### Toggle Role Status

**DELETE /roles/{id}** · Permission: **delete\_role**

- ACTIVE → DISABLED: All users assigned this role lose its permissions immediately.
- DISABLED → ACTIVE: All permissions restored instantly — no reassignment needed.
- Permissions are never deleted from the role — they are ignored at runtime only.

## 6. Managing Permissions

---

### Get All Permissions

**GET /permissions** · Permission: view\_role

Returns all permissions with their ID, name, guardName, and current status.

### Toggle Permission Status

**DELETE /permissions/{id}** · Permission: view\_role

- ACTIVE → DISABLED: All users lose this permission immediately, whether direct or role-based.
- DISABLED → ACTIVE: Permission restored instantly with no reassignment.
- Assignments are never removed — only ignored at runtime when disabled.



## 7. API Endpoint Reference

### Authentication

| Method | Endpoint          | Permission      | Description                           |
|--------|-------------------|-----------------|---------------------------------------|
| POST   | /api/auth/login   | Public          | Login and get access + refresh tokens |
| POST   | /api/auth/logout  | Authenticated   | Revoke refresh token and clear cookie |
| POST   | /api/auth/refresh | Public (cookie) | Get new access token using cookie     |

### Users

| Method | Endpoint    | Permission  | Description                             |
|--------|-------------|-------------|-----------------------------------------|
| POST   | /users      | create_user | Create a new user                       |
| GET    | /users      | view_users  | Get all users (paginated)               |
| GET    | /users/{id} | view_user   | Get user by ID                          |
| PUT    | /users/{id} | update_user | Update user details, roles, permissions |
| DELETE | /users/{id} | delete_user | Toggle user ACTIVE / DISABLED status    |

### Roles

| Method | Endpoint    | Permission  | Description                          |
|--------|-------------|-------------|--------------------------------------|
| GET    | /roles      | view_role   | Get all roles with permissions       |
| GET    | /roles/{id} | view_role   | Get role by ID                       |
| POST   | /roles      | create_role | Create new role with permissions     |
| PUT    | /roles/{id} | edit_role   | Update role name or permissions      |
| DELETE | /roles/{id} | delete_role | Toggle role ACTIVE / DISABLED status |

### Permissions

| Method | Endpoint          | Permission | Description                                |
|--------|-------------------|------------|--------------------------------------------|
| GET    | /permissions      | view_role  | Get all permissions with status            |
| DELETE | /permissions/{id} | view_role  | Toggle permission ACTIVE / DISABLED status |

# 8. Error Responses

All errors follow this structure:

```
{
  "success": false,
  "message": "Error description here",
  "content": null
}
```

| HTTP Status               | Meaning                                              |
|---------------------------|------------------------------------------------------|
| 400 Bad Request           | Invalid input or validation failure                  |
| 401 Unauthorized          | Token expired or invalid — login or refresh required |
| 403 Forbidden             | Valid token but missing required permission          |
| 404 Not Found             | Resource not found                                   |
| 409 Conflict              | Duplicate email or role name                         |
| 500 Internal Server Error | Unexpected server error — check application logs     |